computers

MDPI

Article

ELFA-Log: Cross-System Log Anomaly Detection via Enhanced Pseudo-Labeling and Feature Alignment

Xiaowei Zhao [1,2], Kaiwei Guo [3], Mingting Huang [3], Shaojian Qiu [3,*] and Lu Lu [1,*]

[1] School of Computer Science and Engineering, South China University of Technology, Guangzhou 510006, China; zhaoxw@csg.cn
[2] Digital Research Department, China Southern Power Grid, Guangzhou 510663, China
[3] College of Mathematics and Informatics, South China Agricultural University, Guangzhou 510642, China; guokaiwei@insoft-lab.com (K.G.); huangmingting@insoft-lab.com (M.H.)
* Correspondence: qiushaojian@scau.edu.cn (S.Q.); lul@scut.edu.cn (L.L.)

Abstract

Existing log-based anomaly detection methods typically require large volumes of labeled data for training, presenting significant challenges when applied to new systems with limited labeled data. This limitation has spurred the need for cross-system log anomaly detection (CSLAD) methods. However, current CSLAD approaches often face challenges in effectively handling distributional differences in log data across systems. To address this issue, we propose ELFA-Log, a transfer learning-based approach for cross-system log anomaly detection. By enhancing pseudo-label generation with uncertainty estimation and feature alignment, ELFA-Log improves detection performance even in the presence of data distribution shifts. It uses entropy-based metrics to generate high-confidence pseudo-labels, minimizing reliance on labeled data. Additionally, a distance-based loss function optimizes the shared representation of cross-system log features. Experimental results on benchmark datasets demonstrate that ELFA-Log enhances the performance of CSLAD, offering a practical solution to the challenge of high labeling costs in real-world applications.

Keywords: log anomaly detection; transfer learning; pseudo-labeling; uncertainty estimation



Academic Editor: Xiaochen Lu

Received: 2 June 2025
Revised: 29 June 2025
Accepted: 3 July 2025
Published: 10 July 2025

Citation: Zhao, X.; Guo, K.; Huang, M.; Siu, Q.; Lu, L. ELFA-Log: Cross-System Log Anomaly Detection via Enhanced Pseudo-Labeling and Feature Alignment. *Computers* **2025**, *14*, 272. https://doi.org/10.3390/computers14070272

Copyright: © 2025 by the authors. Licensee MDPI, Basel, Switzerland. This article is an open access article distributed under the terms and conditions of the Creative Commons Attribution (CC BY) license (https://creativecommons.org/licenses/by/4.0/).

1. Introduction

As software systems grow larger and more complex, the likelihood of failures increases [1,2]. Recently, with the increasing importance of online services such as cloud computing, network platforms, and remote databases, ensuring the reliability and stability of these systems has become crucial [3]. Even minor anomalies in system behavior can cause significant disruptions, affecting user experience and potentially leading to substantial financial losses [4–6]. For instance, Alibaba Cloud loses billions of dollars annually due to Intermittent Slow Queries [7], and Amazon has reported that every 0.1-s delay in database loading due to abnormalities results in an additional 1% in financial losses. Consequently, detecting anomalies in logs and mitigating the impact of system failures have become top priorities.

System logs, which capture important events and the system's state during operation, are a vital data source for anomaly detection. Currently, most log-based anomaly detection models rely heavily on the Loghub [8] dataset. Several models have been developed based on Loghub, broadly categorized into supervised [9–11] and unsupervised [12,13] learning methods. However, traditional supervised learning techniques [14–16] depend on large amounts of labeled data, which is often difficult to obtain, especially for newly deployed

systems. On the other hand, unsupervised learning methods [17–19], such as DeepLog [17] and LogBERT [19], have gained wide adoption. These models capture normal patterns from vast quantities of normal logs and use these patterns to identify anomalies. While they have shown promising results, collecting sufficient training data for new systems remains time-consuming and often impractical.

In real-world scenarios, new systems often encounter the "cold start" problem, where a lack of sufficient data hinders the development of effective anomaly detection models. These systems urgently require anomaly detection to ensure stable operation and prevent potential failures. However, due to the scarcity of labeled data—especially when the target system has not yet encountered sufficient anomalous events, traditional unsupervised learning methods are often ineffective. While these methods attempt to identify anomalies by learning normal behavior patterns, their performance is significantly constrained in data-scarce environments.

Facing this limitation, Cross-System Log Anomaly Detection (CSLAD) methods, such as MetaLog [20] and LogTAD [21], have been proposed. These methods aim to enhance anomaly detection in the target system by leveraging knowledge from the source system. However, existing CSLAD methods still face several challenges. First, there are significant differences in structure, log formats, and data distributions between the source and target systems. As a result, current methods struggle to adapt effectively to the target system. Specifically, models in the source system are often trained on specific log features and behavior patterns. This makes it difficult for them to capture the unique patterns in the target system. Additionally, even when data from the source system is used to assist in anomaly detection in the target system, the differences between the systems may still lead to incomplete knowledge transfer [22]. This issue becomes more pronounced when the target system includes log features or behavior patterns not present in the source system. As a result, the loss of information during the transfer process can reduce the accuracy and robustness of detection. Therefore, achieving both accurate and efficient anomaly detection in newly deployed systems remains a key challenge for CSLAD methods.

To address these challenges, we propose ELFA-Log, a transfer learning-based method for cross-system log anomaly detection using enhanced pseudo-labeling with uncertainty estimation and feature alignment. Specifically, ELFA-Log generates high-confidence pseudo-labels from highly certain predictions and quantifies prediction uncertainty using entropy-based measurement. This mechanism effectively filters out low-confidence pseudo-labels, enabling the model to exploit the unlabeled data in the target system fully. As a result, ELFA-Log not only significantly improves detection performance but also effectively mitigates the cold start problem, demonstrating superior capabilities in cross-system log anomaly detection. To further enhance the consistency of cross-system log embeddings, we introduce a distance-based loss function. This function aligns log embeddings from both the source and target systems into a unified feature space by minimizing the deviation of log embeddings from a shared center. By optimizing this alignment, the model better accounts for system differences and effectively leverages both labeled and unlabeled data from the target system, ultimately achieving higher detection accuracy.

Experimental results demonstrate that the proposed method outperforms existing approaches across various cross-system log datasets. It significantly enhances anomaly detection accuracy and robustness, maintaining stable detection performance even in the presence of data imbalance. These findings suggest that our method has substantial practical value and promising application prospects in real-world scenarios.

The following is a summary of our contributions:

- We propose a cross-system log anomaly detection framework that focuses on pseudo-labeling and feature alignment to solve the cold start problem, effectively using both domain and target data.
- This approach enhances detection performance by combining pseudo-labeling with uncertainty measurement and feature alignment, delivering significant improvements, particularly when anomalous data is scarce in the target system.
- Experimental validation on multiple benchmark datasets demonstrates that our method achieves substantial gains in both anomaly detection accuracy and robustness.

The paper is organized as follows: Section 2 reviews related work on log anomaly detection. Section 3 discusses the preliminaries of system logs and the cross-system anomaly detection problem. Section 4 presents our proposed ELFA-Log framework in detail. Section 5 describes the experimental setup, including datasets, baselines, and evaluation metrics. Section 6 presents and analyzes our main experimental findings. These include a comparison with baseline methods, a detailed sensitivity analysis of the model's key hyperparameters, and a series of ablation studies to validate our architectural choices. Following this, Section 7 discusses the threats to the validity of our study and its limitations. Finally, Section 8 concludes the paper and summarizes future research directions.

2. Related Work

Log-based anomaly detection methods are generally classified into unsupervised and supervised categories. Unsupervised methods typically employ RNNs or pre-trained models to learn the features of normal log sequences [23,24]. For instance, DeepLog and LogAnomaly [25] use LSTMs to predict the next log event for anomaly detection, while A2Log [26] leverages attention mechanisms and the Transformer framework to distinguish anomalies through scoring and thresholding. Although unsupervised methods do not rely on labeled data, supervised methods typically achieve higher accuracy when labeled data is available.

Supervised methods are primarily based on classification models, such as LogRobust [9] and LogTransfer [27], which incorporate attention mechanisms and LSTMs to capture cross-system sequence features. PLELog [28] further improves the learning of log sequence features by using an attention-based GRU network. However, supervised methods generally require large amounts of labeled data, which can significantly challenge practical applications.

Many methods construct detection models in log anomaly detection by parsing logs into event sequences. Some approaches use graph models for comparative detection, while others leverage deep learning models to capture the features of log events, such as DeepLog and LogAnomaly. Additionally, the semi-supervised method proposed by PLELog combines unsupervised clustering to infer labels and build anomaly detection models.

For cross-system log anomaly detection, transfer learning methods like LogTAD [21] facilitate knowledge transfer by sharing neural network architectures, helping to bridge the distributional gap between source and target systems. However, when the distribution gap is large, these methods may experience performance limitations. MetaLog [20] introduces meta-learning to improve the model's adaptability to new systems through multi-system meta-training, mitigating distributional issues. Despite the significant progress these methods have made in cross-system log anomaly detection, challenges persist due to insufficient labeled data and large inter-system differences.

To address these challenges, methods that enhance pseudo-label generation with uncertainty measurement are more effective at leveraging unlabeled data. By aligning log embeddings with high-confidence pseudo-labels and incorporating a distance-based

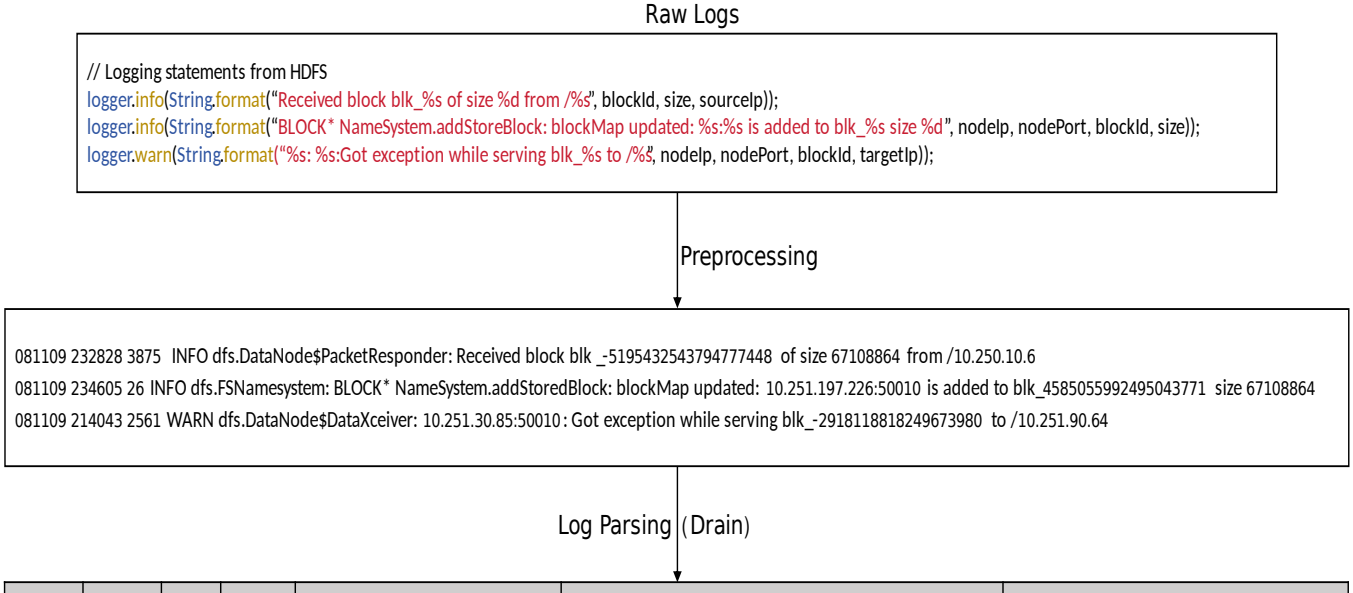
loss function, the robustness and performance of cross-system anomaly detection are further enhanced.

This paper focuses on two key aspects: (1) transferring knowledge from the source system to the target system and (2) addressing the challenge of limited labeled data in the target system through pseudo-labeling, uncertainty measurement, and feature alignment techniques. In Section 4, we provide a detailed description of the proposed method and demonstrate how these approaches enable effective cross-system log anomaly detection.

3. Preliminaries

3.1. System Logs and Log Parsing

System logs are critical records generated by various system components, such as operating systems, applications, and network devices. They track a wide range of activities, including events, errors, warnings, and status updates, and are typically stored in text format. These logs are essential for system monitoring and troubleshooting, but their raw form is often unstructured or semi-structured, making cross-system anomaly detection a complex task. To address this challenge, log parsing techniques are employed to transform raw logs into structured log template sequences. By abstracting recurring patterns into a standardized format, log parsing simplifies log data processing, enabling similar entries to be grouped into categories. For example, Figure 1 illustrates how the Drain method transforms a raw HDFS log into a corresponding log template sequence. Capturing the features of these template sequences allows for the construction of deep neural network-based anomaly detection models, significantly improving the efficiency and accuracy of cross-system log anomaly detection.



Data	Time	Pid	Level	Component	Template	Parameters
081109	232828	3875	INFO	dfs.DataNode$PacketResponder	Received block blk_<*> of size <*> from /<*>	-5195432543794777448, 67108864, 10.250.10.6
081109	234605	26	INFO	dfs.FSNamesystem	BLOCK* NameSystem.addStoredBlock: blockMap updated: <*>:<*> is added to blk_<*> size <*>	10.251.197.226, 50010, 4585055992495043771, 67108864
081109	214043	2561	WARN	dfs.DataNode$DataXceiver	<*>:<*>:Got exception while serving blk_<*> to /<*>	10.251.30, 50010,-2918118818249673980, 10.251.90.64

Figure 1. An example of an HDFS log and the corresponding log template event generated using the Drain log parsing method, where the variables in the log are replaced by placeholders <*> according to the template order.

Effective anomaly detection relies on the accurate interpretation of raw log messages, which are generated by logging systems to capture events within software systems. Each log consists of a header and a content section: the header includes key information such as the log framework, log level, timestamp, and node details, while the content section contains constants and variables. Constants define the event template, while variables represent the dynamic parameters in the log message. Log parsing, a crucial step in automated log analysis plays a vital role in extracting valuable information from semi-structured logs. Its primary objective is to interpret log messages from various systems, applications, and devices, transforming them into structured log event templates for anomaly detection and fault diagnosis tasks.

Traditional log parsing methods rely on manually written regular expressions to extract event templates and key parameters. However, this approach is not only time-consuming but also prone to errors. To address these limitations, researchers have developed various automated log parsing techniques. For instance, Spell [29] uses a data structure based on the longest common subsequence (LCS) to store log keys and associated information. While it achieves high parsing accuracy, it faces limitations in log partitioning due to its reliance solely on LCS. In our work, we employ Drain [30], an efficient log parsing method, to automatically generate log templates from raw log messages.

The core of Drain's methodology is its use of a fixed-depth parsing tree to represent and organize log events. When a new raw log message is processed, it is first tokenized into a sequence of words. Drain then traverses the parsing tree from the root, selecting a path based on the tokens. A key aspect of this process is the application of specific rules at each node; for instance, tokens containing only digits are immediately replaced with a generic wildcard placeholder (<*>), allowing the model to abstract away dynamic parameters. If the log message's token sequence matches an existing path to a leaf node, it is grouped with the templates stored there. If no exact path exists, Drain uses a similarity metric to compare the log message to existing templates at a certain depth. If the similarity is above a predefined threshold, a new template is created by masking the differing tokens. This tree-based approach enables Drain to efficiently and accurately separate the constant part of a message (the event template) from the variable part (the parameters).

The choice of Drain is motivated by its demonstrated high accuracy and efficiency, which is crucial for the reliability of our downstream anomaly detection task. Experimental results from its original study show that Drain performs exceptionally well across multiple system logs, achieving high precision on 16 datasets, including HDFS [31], BlueGene/L [32], and the OpenStack cloud platform [8]. This high parsing accuracy, particularly on the HDFS dataset used in our study, ensures that the log instances are correctly represented as template sequences. By leveraging a robust parser like Drain, we generate high-quality log template sequences, as exemplified in Figure 1, providing a solid and reliable foundation for our anomaly detection model. This transformation is a critical first step, simplifying the processing of complex log data and enhancing the overall validity and accuracy of our detection system.

3.2. Cross-System Log Anomaly Detection

The goal of cross-system log anomaly detection is to connect source and target systems through knowledge transfer, enabling the model to detect anomalies effectively in the target system. However, differences in the structure, semantics, and format of logs generated by different systems present significant challenges. For example, BGL logs capture hardware and low-level system events, HDFS logs document operations in distributed storage systems, and OpenStack logs focus on service and resource management in cloud infrastructure. As shown in Figure 2, examples from these datasets illustrate the diversity

in log formats. These differences make it challenging to directly apply models trained on one system to another, highlighting the need to address such variability and adapt models for target systems.

BGL Logs:
(1) **1117838570** 2005.06.03 R02-M1-N0-C:J12-U11 2005-06-03-15.42.50.363779 R02-M1-N0-C:J12-U11 RAS KERNEL INFO instruction cache parity error corrected
(2) **APPREAD 1117869890** 2005.06.04 R06-M0-N8-I:J18-U01 2005-06-04-00.24.50.681440 R06-M0-N8-I:J18-U01 RAS APP FATAL ciod: failed to read message prefix on control stream (CioStream socket to 172.16.96.116:33216
(3) **1117973923** 2005.06.05 R30-M1-NC-C:J09-U11 2005-06-05-05.18.43.720053 R30-M1-NC-C:J09-U11 RAS KERNEL INFO generating core.6686

HDFS Logs:
(1) **081109** 203615 148 INFO dfs.DataNode$PacketResponder: PacketResponder 1 for block blk_38865049064139660 terminating
(2) **081109** 203807 222 INFO dfs.DataNode$PacketResponder: PacketResponder 0 for block blk_-6952295868487656571 terminating
(3) **081109** 204005 35 INFO dfs.FSNamesystem: BLOCK* NameSystem.addStoredBlock: blockMap updated: 10.251.73.220:50010 is added to blk_7128370237687728475 size 67108864

OpenStack Logs:
(1) nova-api.log.1.2017-05-16_13:53:08 2017-05-16 00:00:00.008 25746 INFO nova.osapi_compute.wsgi.server [req-38101a0b-2096-447d-96ea-a692162415ae 113d3a99c3da401fbd62cc2caa5b96d2 54fadb412c4e40cdbaed9335e4c35a9e - - -] 10.11.10.1 "GET /v2/54fadb412c4e40cdbaed9335e4c35a9e/servers/detail HTTP/1.1" status: 200 len: 1893 time: 0.2477829
(2) nova-compute.log.1.2017-05-16_13:55:31 2017-05-16 00:00:04.562 2931 INFO nova.computer.manager [req-3ea4052c-895d-4b64-9e2d-04d64c4d94ab - - - - -] [instance: b9000564-fe1a-409b-b8cc-1e88b294cd1d] VM Paused (Lifecycle Event)

Figure 2. BGL,HDFS and OpenStack Logs.

Existing methods often use domain-invariant representation learning and transfer learning to address these challenges. Domain-invariant representation learning employs techniques such as adversarial training, contrastive learning, or feature alignment to minimize the distributional differences between the source and target systems. For example, adversarial training utilizes a discriminator to optimize feature extractors, thereby encouraging consistency across systems, while contrastive learning employs sample pairs to capture shared patterns between normal and anomalous behaviors. Transfer learning, on the other hand, involves pretraining on labeled data from the source system and fine-tuning with a small amount of labeled data from the target system to suit its specific characteristics better.

While these approaches help reduce the gap between source and target systems, they face limitations. Transfer learning depends on labeled data from the target system, which is often scarce, particularly for anomaly samples. Domain-invariant representation learning can align feature distributions but may struggle to generalize when the target system exhibits unseen patterns, reducing detection effectiveness.

To address these issues, we propose an approach that combines pseudo-label generation with uncertainty measurement and feature alignment. A model trained on the source system is used to generate pseudo-labels for unlabeled data in the target system, which are then filtered using uncertainty measurement to retain only high-confidence labels. This ensures the reliability of the training data. Additionally, domain-invariant feature alignment is applied further to mitigate differences between the source and target systems. This approach aims to reduce reliance on labeled data from the target system and improve robustness in scenarios with limited data availability.

In summary, our approach builds on existing techniques by incorporating pseudo-labeling and uncertainty filtering to handle data scarcity in the target system. Combined with feature alignment, this method offers a practical approach to enhance cross-system anomaly detection in environments such as distributed storage, cloud computing, and hardware systems.

4. Method

4.1. Framework

This paper presents ELFA-Log, a cross-system log anomaly detection framework that addresses the challenges of distributional differences and insufficient labeled data in the target domain. By enhancing pseudo-labeling [33] with uncertainty measurement techniques [34], the framework improves generalization. It maps log sequences from both systems to a shared feature space, where pseudo-labeling generates high-confidence labels and uncertainty measurement filters out low-confidence predictions. Anomalies are detected based on the distance from the learned center in the low-dimensional space, with sequences farther from the center flagged as anomalies. Figure 3 illustrates the framework, which comprises four key components: log sequence processing, pseudo-labeling with uncertainty measurement, centralization and domain adversarial learning, and anomaly detection. Together, these components enable effective cross-system log anomaly detection by addressing data distribution shifts and minimizing the dependency on labeled data.

We denote the source system as S and the target system as T, with D representing the log data from both systems. The * serves as a generic wildcard placeholder. It is used to replace dynamic or variable parameters within the log messages, such as numerical values or other specific data that changes frequently. The n-th log message sequence generated by the system is represented as x_n. The source system's dataset is represented as $D_s=\{x_i\}_{i=1}^{L_s}$, while the target system's dataset is represented as $D_t=\{x_i\}_{i=1}^{L_t}$.

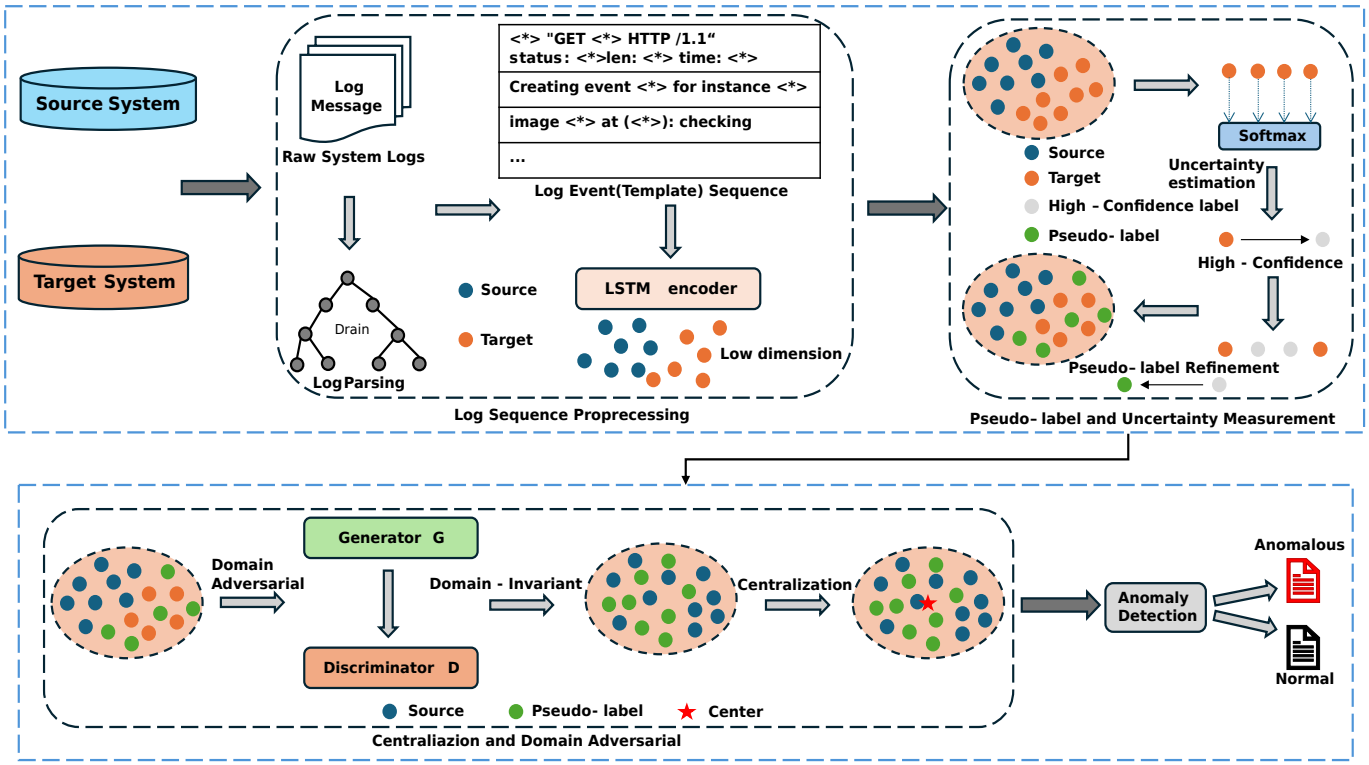


Figure 3. The Framework of ELFA-Log.

4.2. Log Sequence Processing

In this study, the first step is to parse the raw log messages into structured, machine-readable data. For this, we use the Drain method, which efficiently extracts key events and attributes from the logs, ensuring that critical information is captured accurately. After parsing, we serialize the log data by converting each log event into a fixed-length sequence. This standardization helps to prepare the data for further processing. We then apply the word2vec [35] algorithm to transform each serialized log entry into an embedding vector.

This allows the model to capture the complex relationships between log events, such as similar error patterns or recurring operational sequences.

To account for the temporal dependencies between log messages, we use an LSTM (Long Short-Term Memory) network to encode the log sequence. The LSTM processes each log message using the following equation:

$$h_n = LSTM(x_n, h_n - 1),$$

where x_n is the feature vector of the n-th log message, and h_n is the hidden state of that message. This process maps the entire log sequence into a low-dimensional space, effectively capturing both the context and the temporal dependencies between log events.

By combining the Drain method, word2vec, and LSTM, we are able to convert the raw log data into structured embeddings. This process captures the intricate relationships and temporal patterns within the logs, providing a solid foundation for further tasks such as log analysis, anomaly detection, and predictive modeling.

4.3. Pseudo-Labeling and Uncertainty Measurement

After the LSTM encoder processes the log data and generates the corresponding hidden states, the next step is to compute the predicted probability and entropy values for the target system samples. These values are essential for guiding the model's understanding of the feature distributions of the target system. The predicted probabilities provide insights into the model's confidence in its predictions, while the entropy helps measure the uncertainty of these predictions.

To begin, the input is represented as a two-dimensional tensor p, where each row corresponds to the probability distribution of a sample across different classes. The model generates this distribution after processing the log data through the LSTM encoder. Each element of the tensor p represents the probability that a given sample belongs to a specific class. With this tensor, we can compute the entropy of each probability distribution, which provides a measure of uncertainty in the model's predictions. The entropy is calculated using the following formula:

$$H(P) = -\sum_{i=1}^{n} p_i \log(p_i)$$

where p_i is the probability of the i-th class, and n is the total number of classes. The entropy function quantifies the unpredictability of the system's output: a lower entropy value indicates higher confidence in the model's predictions, while a higher entropy value suggests greater uncertainty. To avoid issues with taking the logarithm of zero, we add a small positive value ϵ to the probability terms (e.g., 1×10^{-5}). Once the entropy for each sample is computed, the feature vectors generated by the LSTM encoder are converted into probability distributions using the softmax function:

$$prob_i(j) = \frac{e^{features_i(j)}}{\sum_k e^{features_i(k)}}$$

This step transforms the raw feature vectors into a normalized probability distribution over all classes, where $prob_i(j)$ represents the probability that the i-th sample belongs to class j.

Algorithm 1 formally describes the pseudo-label generation process. After the probability distribution is obtained (line 8), we first compute the maximum probability and its corresponding class index for each sample (line 9). When uncertainty measurement is enabled (line 10), we calculate the information entropy for each probability distribution

(line 11) and apply a dual-threshold filter (line 11). The mask identifies samples where the model's predictions are highly confident and certain. The generated masks are defined as follows:

$$\text{mask}_i = \begin{cases} \text{true, if } \max_probs_i > \text{threshold and } H(\text{probs}_i) < 1.0 \\ \text{false, otherwise} \end{cases}$$

Algorithm 1 Pseudo-Label Generation

Require:
1: $features$: Hidden states from LSTM encoder, shape (B, D)
2: threshold: Confidence threshold $\in [0, 1]$
3: ϵ: Small constant for numerical stability (10^{-5})
Ensure:
4: $pseudo_labels$: Predicted class indices, shape $(B,)$
5: $mask$: Boolean selection mask, shape $(B,)$
6: Disable gradient computation
7: $logits, _ \leftarrow \text{Encoder}(features, \alpha = 0)$
8: $probs \leftarrow \text{Softmax}(logits, \text{dim} = 1)$
9: $\max_probs, pseudo_labels \leftarrow \text{Max}(probs, \text{dim} = 1)$
10: **if** using uncertainty measurement **then**
11: $mask \leftarrow (\max_probs \geq \text{threshold}) \wedge (entropy < 1.0)$
12: **else**
13: $mask \leftarrow \max_probs \geq \text{threshold}$
14: **end if**
15: **return** $pseudo_labels, mask$

In our method, $\max_probs_i$ represents the highest predicted probability for the i-th sample, and $H(\text{probs}_i)$ is the sample's information entropy. A sample is considered a high-confidence prediction if it meets two conditions: its $\max_probs_i$ exceeds a predefined confidence threshold, and its associated entropy is low.

The threshold for low entropy is set to 1.0, and this choice is theoretically motivated. For a binary classification task like ours (normal vs. anomaly), the information entropy of a probability distribution ranges from 0 (perfect certainty) to a maximum of 1.0. This maximum value corresponds to a state of complete uncertainty where the model predicts a probability of [0.5, 0.5], effectively guessing. Therefore, our criterion of $H(\text{probs}_i) < 1.0$ serves as a principled filter to exclude only these most ambiguous predictions. Next, pseudo-labels are generated based on these high-confidence samples (i.e., samples with high maximum probabilities and low entropy). These pseudo-labels are added to the target system's dataset, thereby enhancing the diversity of the training data and helping the model train more effectively in scenarios with limited data.

This approach helps the model focus on the samples it is most confident about, thereby improving the accuracy of the predictions and enhancing its understanding of the target system's feature distribution.

4.4. Centralization and Domain Adversarial

In this study, we leverage both centralization and domain adversarial learning methods to improve the representation of log sequences from different systems. The aim is to ensure that the representation of normal log sequences is concentrated around a central value, c, and that log sequences from different systems are mapped to a shared feature space through domain adversarial learning. This approach enables the model to learn generalized features that apply to both the source and target systems, thereby enhancing the model's ability to adapt to new, unseen systems.

The centralization step is crucial in this process. To begin, we compute the center value for the source system's log sequences. This center, denoted as c_i, represents the average feature vector of the normal log sequences in the source domain. In the case of the target system, since labeled data may not be available, we associate the log sequences with pseudo-labels. The centralization process involves calculating the average of all center values from the source system, which gives us the global center c for both the source and target systems. This is mathematically expressed as: i.e., $c = \frac{1}{n}\sum_{i=1}^{n} c_i$.

To make normal log data closer to the center while pushing abnormal logs farther from the center, we define the following loss function:

$$L = \frac{1}{N}\sum_{i=1}^{N} d(x_i, c)$$

where x_i represents the feature vector of the i-th log sequence, and $d(x_i, c)$ is the distance between the log sequence and the center. N is the total number of log sequences in the dataset. This loss function encourages the model to bring normal log sequences closer to the center while increasing the distance for abnormal log sequences, which helps to separate normal and abnormal events in the feature space.

Next, we perform domain adaptation in the framework, which includes two key components: the generator G and the discriminator D. The generator G is a shared LSTM network that maps log sequences into a common feature space. The goal of the discriminator D is to distinguish whether a feature vector comes from the source domain or the target domain. The output of the discriminator D is represented as:

$$D(v) = \text{Sigmoid}(w \cdot v + b)$$

where v is the feature vector input to the discriminator, w is the weight vector, and b is the bias term. The sigmoid function ensures that the output of the discriminator is a value between 0 and 1, where a value close to 1 indicates that the feature vector is likely from the source domain, and a value close to 0 indicates that it is likely from the target domain.

The training objective of the domain adaptation model is to make the feature vectors generated by the generator G indistinguishable to the discriminator D. This means that the generator learns to produce feature vectors that are equally plausible as coming from either the source or the target system, thus facilitating domain adaptation. To achieve this, we implement an adversarial loss function $\mathcal{L}_{\text{adv}}$ that penalizes the discriminator for distinguishing between source and target samples. The adversarial loss pushes the generator to create feature vectors that are domain-invariant, making the model more robust and adaptable to new systems.

Finally, the total loss function used in training combines two key components: the reconstruction loss of the generator and the adversarial loss. The reconstruction loss ensures that the generated feature vectors are accurate representations of the original log data, while the adversarial loss encourages the generator to produce domain-invariant representations. The total loss function is expressed as: $\mathcal{L}_{\text{all}} = \mathcal{L}_{\text{G}} + \mathcal{L}_{\text{adv}}$.

4.5. Anomaly Detection

After training, the generator G is used to extract feature representations from the log sequences and aligned feature for effective cross-system detection. The Euclidean distance between each extracted feature vector v and the predefined center c in the learned feature space is then computed. If the distance exceeds a pre-set threshold p, the log sequence is flagged as an anomaly. This is formalized as:

$$\text{Anomaly}(v) = \begin{cases} 1, & \text{if } \|v - c\|_2 > p \\ 0, & \text{otherwise} \end{cases}$$

where $\|v - c\|_2$ represents the Euclidean distance between the feature vector v and the center c, and p is the threshold that determines the boundary between normal and anomalous log sequences.

The threshold p plays a key role in the performance of the anomaly detection model. A smaller threshold will lead to more sequences being identified as anomalies, increasing the false positive rate. Conversely, a larger threshold may miss some anomalies, resulting in a higher false negative rate. Thus, p needs to be carefully tuned to achieve an optimal balance. This tuning can be performed through techniques like cross-validation on labeled datasets, or by using unsupervised methods such as clustering to estimate the natural variance of normal log sequences and set a reasonable threshold.

Once anomalies are detected, they can trigger further analysis or activate real-time alerts, allowing for timely intervention. By using the distance from a learned center to identify anomalies, this method is capable of detecting both major deviations and minor irregularities in the log data, thereby increasing the overall effectiveness and reliability of the detection system.

5. Experimental Setup

5.1. Datasets

We evaluated the proposed method on three widely-used public benchmark datasets: HDFS (v1), BGL, and OpenStack, all sourced from the LogHub repository [8]. Table 1 provides a summary of the key characteristics of these datasets, including the total number of log messages and the number of anomalous entries. These datasets were selected because they represent different types of systems—HDFS, a distributed file system; BGL, a high-performance computing environment; and OpenStack, a cloud platform. By using datasets from these varied systems, we aim to assess the method's performance across different log structures and system types. This helps to provide a more comprehensive evaluation of the proposed approach.

- BlueGene/L (BGL). The BGL dataset is sourced from the BlueGene/L supercomputer at Lawrence Livermore National Laboratory, containing 4,747,963 log messages, with 348,460 marked as anomalies. For the analysis, we used a sliding window approach to construct log sequences from the raw data.
- Hadoop Distributed File System (HDFS). The HDFS dataset was generated by running Hadoop-based MapReduce jobs on Amazon EC2 nodes, resulting in a total of 11,172,157 log messages. Due to the large size of the original HDFS data, we selected a subset of 5,000,000 log messages for our evaluation, with 284,818 labeled as anomalies. Each log message includes a session ID, which allows us to group related log entries into sequences for analysis.
- OpenStack Dataset. The OpenStack dataset consists of logs generated by various cloud infrastructure services, including compute, storage, and networking components. We collected around 70,724 log messages, with 18,435 identified as anomalies. These logs represent real-world cloud platform activities and are useful for evaluating anomaly detection in cloud environments.

Table 1. Statistics of the datasets.

Dataset	Log Messages	Anomalies	Description
BGL	4,747,963	348,460	The BGL dataset is a labeled collection of system logs from the Blue Gene/L supercomputer, featuring a high density of anomalies, ideal for anomaly detection research.
HDFS	5,000,000	284,818	The HDFS dataset comprises labeled system logs from the Hadoop Distributed File System, characterized by a diverse range of anomalies, making it suitable for comprehensive anomaly detection and analysis in distributed computing environments.
OpenStack	70,724	18,435	The OpenStack dataset includes labeled log messages from the OpenStack cloud platform, offering a smaller but diverse set of error types in cloud service environments, useful for studying and developing anomaly detection techniques in cloud computing.

For all datasets, we first use Drain to parse the raw log messages into structured logs, which are then converted into sequences. Next, we retain a portion of the normal log sequences for training, while the remaining normal logs and all anomalous logs are used for validation and testing.

5.2. Baselines

We compare the proposed model with the following baseline models, which represent recent advancements in cross-system log anomaly detection:

- LogTAD [21]. LogTAD is a deep learning model for cross-system log anomaly detection. It learns patterns from log sequences across multiple systems, clusters normal sequences around a shared center, and identifies anomalies by measuring their distance from this center. This method has demonstrated effectiveness in handling variations across systems.
- MetaLog [20]. MetaLog is a meta-learning framework designed specifically for cross-system log anomaly detection. It employs a meta-training strategy to quickly adapt to new systems with limited labeled data in the target system. By learning transferable features from multiple source domains, MetaLog enhances its ability to detect anomalies in diverse environments. This framework is a recent development aimed at improving cross-system adaptability.

These baseline models were chosen because they are state-of-the-art methods developed in recent years, specifically addressing the challenges of cross-system log anomaly detection. Their methodologies align closely with our task, providing a strong basis for comparison.

5.3. Implementation Details

The raw log data is first parsed into structured logs using the Drain method, which efficiently transforms unstructured log events into a format that is more amenable to analysis. These structured logs are then serialized into fixed-length sequences to ensure uniformity across inputs and enable better processing by machine learning models. Once serialized, each log event is mapped to a continuous vector space using the word2vec [35] algorithm, which captures the semantic relationships between different log events. This approach helps to represent logs in a way that preserves their meaning, ensuring that similar events are represented by similar vectors.

Domain adaptation techniques, such as adversarial training and pseudo-labeling, are applied to align source and target domain log data. Adversarial training uses a dis-

criminator to encourage the model to learn features that are consistent across domains. Pseudo-labeling allows the model to generate additional informative data based on its own predictions, improving performance by leveraging both domains more effectively.

Several key hyperparameters are optimized during training, including the learning rate, LSTM dimensions, batch size, and other model-specific parameters. These hyperparameters are tuned using the validation set to achieve optimal model performance. Specific settings include a 128-dimensional hidden state for the LSTM, a word2vec embedding size of 300, a batch size of 1024, and up to 100 training epochs with early stopping to prevent overfitting. Additionally, the model utilizes a sliding window size of 20 and a step size of 4 for data preprocessing. For anomaly detection, sequences whose distance from the learned center c exceeds a predefined threshold p are flagged as anomalies, enabling the model to detect outlier log sequences that significantly deviate from normal behavior.

5.4. Evaluation Metric

Log anomaly detection is typically framed as a classification problem, where the goal is to distinguish between normal and anomalous log sequences. To evaluate model performance, standard classification metrics such as Precision, Recall, and F1-score are commonly used. These metrics provide a comprehensive assessment of the model's effectiveness in anomaly detection and error minimization, and are defined as follows:

$$Precision = \frac{TP}{TP + FP}$$

$$Recall = \frac{TP}{TP + FN}$$

$$F1 - score = \frac{2 * Precision * Recall}{Precision + Recall}$$

$$MCC = \frac{(TP \times TN) - (FP \times FN)}{\sqrt{(TP + FP)(TP + FN)(TN + FP)(TN + FN)}}$$

where:

- TP (True Positive) refers to the number of anomalies that are correctly identified as anomalies.
- FP (False Positive) represents the number of normal log sequences incorrectly classified as anomalies.
- FN (False Negative) refers to the number of anomalies that are mistakenly classified as normal.
- TN (True Negative) represents the number of normal log sequences that are correctly identified as normal.

The F1-score and Matthews Correlation Coefficient (MCC) are two key metrics for evaluating model performance. The F1-score is the harmonic mean of Precision and Recall, providing a single value that balances both. It is particularly helpful when managing the trade-off between Precision and Recall, as improving one can often reduce the other. A higher F1-score, closer to 1, indicates a model that effectively balances false positives and false negatives. By giving equal importance to Precision and Recall, the F1-score serves as a reliable measure when the costs of false positives and false negatives are similar.

MCC is another important metric that evaluates performance across all four components of the confusion matrix: true positives, true negatives, false positives, and false negatives. It is especially effective in handling imbalanced datasets, where traditional metrics may fall short. MCC values range from -1 to 1, with 1 indicating perfect predictions and -1 showing completely incorrect predictions. Unlike the F1-score, which focuses on

Precision and Recall, MCC measures the overall quality of the predictions, making it ideal for tasks with imbalanced data or uneven error costs.

In log anomaly detection, balancing Precision, Recall, F1-score, and MCC is essential. False positives can lead to unnecessary alerts, wasting resources, and reducing operator sensitivity to real problems. False negatives, where actual anomalies go undetected, can lead to system failures or security breaches with severe consequences. The F1-score minimizes false positives and false negatives in a balanced way, while MCC provides a broader evaluation of the model's predictive performance. By employing both metrics, we gain a more complete and reliable picture of a model's performance in terms of its reliability, robustness, and effectiveness. Such a thorough approach is vital in real-world applications where error costs are significant.

6. Experimental Result

6.1. Main Result

To thoroughly evaluate the effectiveness of the proposed model, we conducted extensive log anomaly detection experiments using three widely used datasets: BGL, HDFS, and OpenStack. The experiments were carried out across six cross-system scenarios: BGL → HDFS, HDFS → BGL, BGL → OpenStack, OpenStack → BGL, HDFS → OpenStack, and OpenStack → HDFS. In each scenario, the dataset before the arrow represents the source system, and the dataset after the arrow represents the target system (e.g., BGL → HDFS means BGL is the source system, and HDFS is the target system). This setup is intended to evaluate the model's performance across different systems, reflecting scenarios where models are required to adapt to new and diverse system environments.

In this paper, we employ both F1-score and Matthews Correlation Coefficient (MCC) as evaluation metrics to comprehensively assess the performance of our model. F1-score effectively reflects the model's ability to handle false positives and false negatives by balancing precision and recall. On the other hand, MCC evaluates performance across true positives, false positives, true negatives, and false negatives, making it particularly suitable for scenarios with imbalanced data distributions. These two metrics complement each other, providing a more thorough and reliable evaluation framework for our cross-system log analysis tasks. Table 2 compares the performance of our model with the baseline methods across six cross-system scenarios.

In the table, bold text indicates the best performance, while underlined text indicates the second-best performance, providing a clear visualization of the performance differences among the models in various scenarios. This annotation further highlights the significant performance advantage of our model in most scenarios, as well as the subtle differences in certain cases. While LogTAD and MetaLog perform well in certain cases, our model consistently shows improvements across most scenarios, demonstrating efficient generalization capabilities. In particular, in the HDFS → BGL and HDFS → OpenStack scenarios, our model achieves notably higher F1-score and MCC compared to the baseline methods. Additionally, it delivers better average performance across all scenarios, highlighting its robustness and adaptability across different system environments.

To further visualize the performance distribution and stability of each method, Figure 4 presents a box plot of the F1-scores for each method to show their performance distribution across the six tasks. The median F1-score of ELFA-Log is higher than that of MetaLog and LogTAD. ELFA-Log demonstrates more consistent performance across tasks, as indicated by the shorter box in its plot compared to MetaLog.

Table 2. Experimental Results.

Metrics	**F1-Score**			**MCC**		
Method	**LogTAD**	**MetaLog**	**ELFA-Log**	**LogTAD**	**MetaLog**	**ELFA-Log**
BGL → HDFS	64.23	**81.22**	71.83	0.486	**0.723**	0.674
HDFS → BGL	70.65	80.04	**87.61**	0.417	0.514	**0.814**
BGL → OpenStack	53.19	46.63	**58.43**	0.043	0.047	**0.054**
OpenStack → BGL	79.51	86.89	**87.30**	0.596	**0.806**	0.721
HDFS → OpenStack	54.68	42.14	**57.42**	0.026	0.043	**0.085**
OpenStack → HDFS	62.87	62.31	**64.82**	0.309	0.276	**0.364**
AVG	64.18	66.53	**71.23**	0.312	0.401	**0.452**

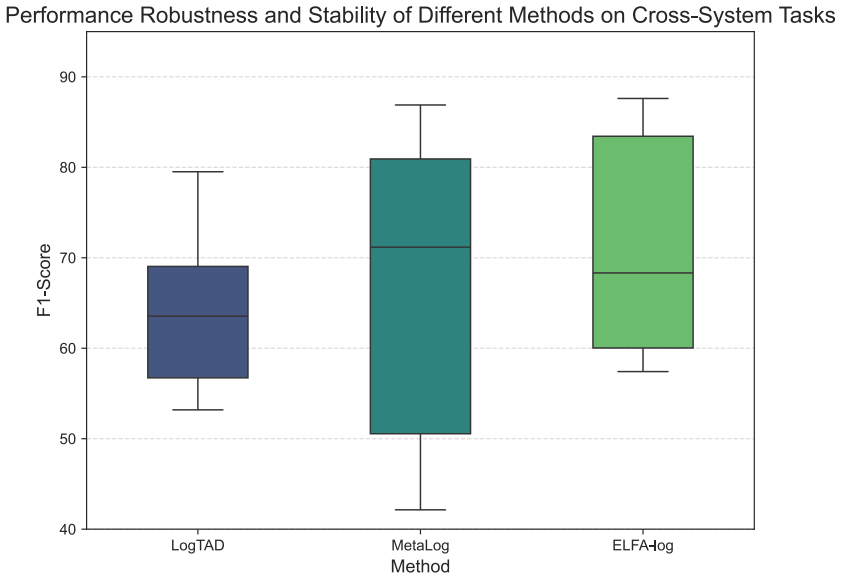


Figure 4. Performance Robustness and Stability of Different Methods on Cross-System Tasks. The box plot shows the distribution of F1-scores for each method across all six cross-system scenarios.

The numerical results in Table 2 reflect these observations. For instance, in the HDFS → BGL task, ELFA-Log achieves an F1-score of 87.61, compared to 70.65 for LogTAD and 80.04 for MetaLog. In the same scenario, the MCC for ELFA-Log is 0.814, while the values for LogTAD and MetaLog are 0.417 and 0.514, respectively.

The success of our model can be attributed to its advanced architecture, which integrates several key techniques. The model captures long-term dependencies within log sequences, enabling it to identify complex patterns and irregularities. Moreover, the incorporation of domain adversarial training and pseudo-labeling mechanisms further enhances its ability to generalize across systems. Domain adversarial training encourages the model to learn features that are less sensitive to system-specific variations. At the same time, pseudo-labeling allows the model to generate labels based on its own predictions, improving performance even when labeled data is limited. These mechanisms enable the model to understand normal log behavior better and identify deviations from this norm, even in scenarios with fewer labeled examples in the target system.

In summary, our model demonstrates better performance across all six cross-system scenarios, highlighting its adaptability to diverse environments. By integrating advanced modeling techniques, domain adversarial training, and pseudo-labeling, the model demonstrates improved generalization across different environments. This makes our approach well-suited for real-world log analysis, where system differences exist, and labeled data may be scarce. Our results indicate that integrating these techniques enhances model reliability, providing a promising solution for log analysis in dynamic and complex settings.

6.2. Analysis of Hyperparameter Sensitivity

The selection of an appropriate pseudo-label confidence threshold is a crucial step in striking a balance between the quality and quantity of self-generated labels. A detailed sensitivity analysis was performed for this hyperparameter across all six cross-system tasks to validate the choices made for our main experiments. The findings, presented in Figure 5, illustrate the model's F1-score as a function of the threshold. The analysis reveals that the optimal threshold is highly dependent on the intrinsic characteristics of the target domain.

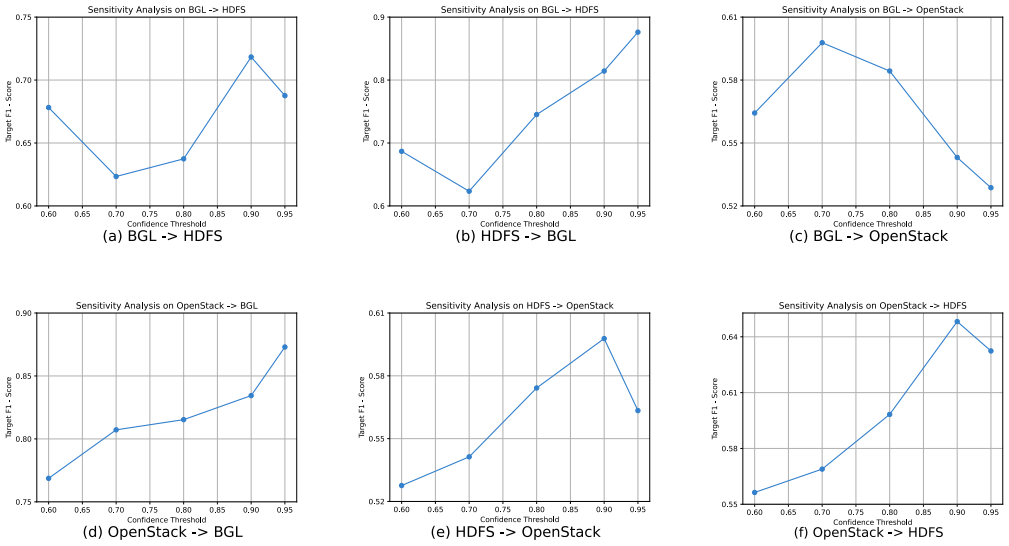


Figure 5. Sensitivity of the F1-score to the pseudo-label confidence threshold across all six domain adaptation scenarios. The results illustrate that no single threshold is optimal for all tasks, highlighting the necessity of adapting this hyperparameter to the specific characteristics of the target domain.

For tasks with HDFS and BGL as the target domains, the analysis in Figure 5 shows that higher confidence thresholds lead to better results, with the best F1-scores achieved at 0.90 and 0.95. This indicates that prioritizing the precision of pseudo-labels is a suitable approach for these scenarios.

Tasks targeting the OpenStack dataset show a different trend, where a lower threshold of 0.8 was more effective. This is because the OpenStack dataset exhibits a more significant class imbalance, resulting in lower prediction probabilities for the anomaly class. A reduced threshold is therefore needed to increase recall by including more potential anomaly predictions in the pseudo-labeling process.

Based on theses sensitivity analysis, we suggest that for practical applications, a confidence threshold within the range of 0.80 to 0.95 can be selected to achieve effective performance across different system environments.

6.3. Ablation Studies

To assess the individual contributions of the pseudo-labeling mechanism and uncertainty measurement method, we conducted a series of ablation experiments. These experiments were designed to systematically remove or disable each component and examine how these changes affected the model's performance across different cross-system scenarios. By quantifying the impact of each mechanism on the overall model performance, we gained insights into the specific roles each component plays.

Figure 6 presents the Precision, Recall, and F1-Score of three model variants across six cross-system scenarios. In each case, the performance of the full model was compared to that of variants where either the pseudo-labeling mechanism or the uncertainty measurement method was omitted. The ablation experiments reveal that both components

contribute to the model's ability to generalize and its detection performance, though the extent of their impact varies across scenarios.

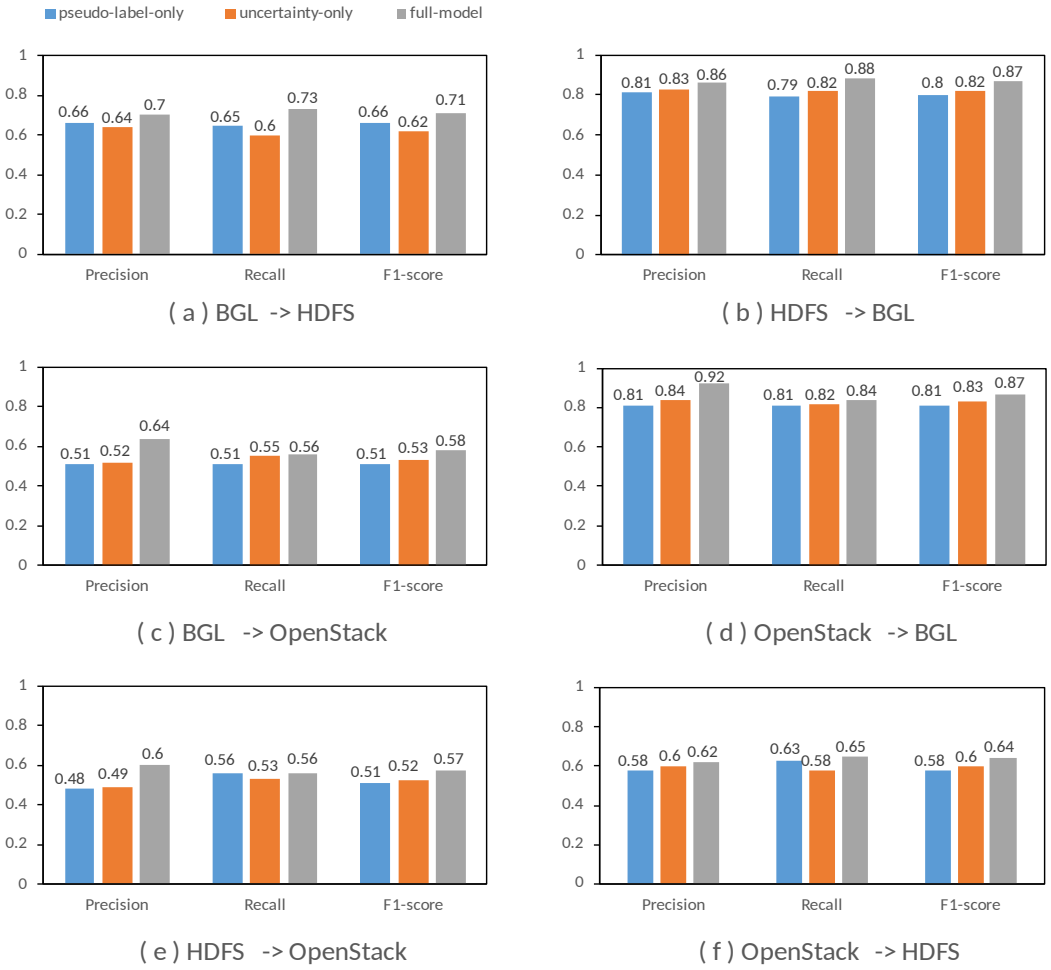


Figure 6. Ablation Studies Results.

For example, when the pseudo-labeling mechanism was removed, the model showed a decline in performance, especially in cases where labeled data was limited. Similarly, the absence of the uncertainty measurement method led to a reduction in model robustness, particularly in distinguishing between normal and anomalous log sequences. These results suggest that both components support the model's performance, particularly when the system must adapt to new environments with limited labeled data.

The findings underscore the value of incorporating both mechanisms into the model. The pseudo-labeling mechanism enables the model to leverage its own predictions to generate pseudo-labels for the target domain, helping mitigate the reliance on labeled data. The uncertainty measurement method enhances the model's decision-making by providing an estimate of prediction confidence, which can be useful in ambiguous cases. Together, these components help the model handle variability across different systems more effectively.

These results suggest that both mechanisms contribute to improving model performance in cross-system scenarios. Future research could explore ways to optimize these components further or experiment with their integration with other techniques to address more complex or data-scarce situations. The findings from these ablation experiments provide valuable insights into how these mechanisms impact model generalization and robustness, which may inform future work in log analysis across diverse, real-world applications.

7. Threats to Validity

This section discusses the limitations of our study and directions for future research.

Internal validity. Regarding the statistical validation of our results, this study provides a box plot analysis (Figure 4) to visualize performance stability. Future work could extend this by conducting repeated trials to perform formal significance tests, such as *t*-tests, and to establish confidence intervals. Furthermore, this study focuses on detection effectiveness, measured by F1-score and MCC, and does not include an analysis of computational efficiency. An evaluation of metrics such as training time and inference latency is an important area for future work to assess the model's practical feasibility.

External validity. The generalizability of our findings is subject to the following considerations. First, the model's transfer performance is dependent on the types of systems and the similarity between domains. Our evaluation across three public benchmarks indicates that performance is higher on transfers between more similar systems (e.g., HDFS and BGL). In contrast, it is comparatively weaker when targeting the structurally divergent OpenStack dataset, which has a notable class imbalance. This suggests the model is most effective when a degree of similarity exists between source and target logs. Second, the effectiveness of the adaptation mechanism relies on a sufficient volume of target data. While the method is designed for data-scarce scenarios, a certain number of unlabeled log sequences is necessary for the model to learn the target distribution. The performance on tiny target datasets was not evaluated in this study, and this remains a boundary condition for its application. Additionally, while our selected baselines are representative, a broader comparative study is warranted. Future work should therefore also evaluate our model against other important classes of methods, including classic sequential models like DeepLog and Transformer-based approaches like LogBERT.

8. Conclusions

This paper proposes a transfer learning framework for cross-system log anomaly detection, addressing distributional differences and the scarcity of labeled data in new environments. Unlike traditional methods hindered by these challenges, our solution enhances pseudo-labeling through uncertainty measurement and feature alignment. Specifically, the framework quantifies prediction uncertainty to identify high-confidence samples for reliable pseudo-label generation, reducing dependency on labeled data. This enhances adaptation to target systems and robustness in the presence of ambiguous logs. Experiments demonstrate sustained high detection performance under significant distribution shifts and scarce labels. The framework significantly lowers manual labeling costs, especially during resource-intensive early deployment phases. Its modular design ensures flexibility to handle log structure variations, noise, and system evolution.

Future work will enhance the framework for complex scenarios and broader applications. We will explore advanced feature extraction to capture finer log patterns, test alternative uncertainty quantification for noisy environments, and evaluate performance across diverse datasets and systems. Extensions can adapt the framework to accommodate other log types, such as network traffic or IoT logs.

Author Contributions: Conceptualization, X.Z. and S.Q.; methodology, X.Z. and K.G.; software, X.Z. and K.G.; validation, K.G. and M.H.; formal analysis, X.Z. and K.G.; investigation, K.G. and M.H.; resources, X.Z. and K.G.; data curation, K.G.; writing—original draft preparation, X.Z. and K.G.; writing—review and editing, M.H., S.Q. and L.L.; visualization, K.G.; supervision, L.L.; project administration, S.Q. and L.L.; funding acquisition, S.Q. and L.L. All authors have read and agreed to the published version of the manuscript.

Funding: This research was funded by the Guangdong Natural Science Fund Project (Grant No. 2024A1515010204), the Guangdong Basic and Applied Basic Research Foundation (No. 2022A1515110564), and the China Southern Power Grid Company Limited (Grant No. ZBKJXM20232483).

Data Availability Statement: The data presented in this study are available on request from the corresponding author.

Conflicts of Interest: Author Xiaowei Zhao was employed by the company China Southern Power Grid. The remaining authors declare that the research was conducted in the absence of any commercial or financial relationships that could be construed as a potential conflict of interest.

References

1. Chen, J.; He, X.; Lin, Q.; Xu, Y.; Zhang, H.; Hao, D.; Gao, F.; Xu, Z.; Dang, Y.; Zhang, D. An empirical investigation of incident triage for online service systems. In Proceedings of the 2019 IEEE/ACM 41st International Conference on Software Engineering: Software Engineering in Practice (ICSE-SEIP), Montreal, QC, Canada, 25–31 May 2019; IEEE: Piscataway, NJ, USA; ACM: New York, NY, USA, 2019; pp. 111–120.
2. Chen, J.; Zhang, S.; He, X.; Lin, Q.; Zhang, H.; Hao, D.; Kang, Y.; Gao, F.; Xu, Z.; Dang, Y.; et al. How incidental are the incidents? Characterizing and prioritizing incidents for large-scale online service systems. In Proceedings of the 2020 35th IEEE/ACM International Conference on Automated Software Engineering (ASE), Melbourne, VIC, Australia, 21–25 September 2020; IEEE: Piscataway, NJ, USA, 2020; pp. 373–384.
3. Jiang, J.; Lu, W.; Chen, J.; Lin, Q.; Zhao, P.; Kang, Y.; Zhang, H.; Xiong, Y.; Gao, F.; Xu, Z.; et al. How to mitigate the incident? An effective troubleshooting guide recommendation technique for online service systems. In Proceedings of the 28th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering, Virtual Event, 8–13 November 2020; ACM: New York, NY, USA, 2020; pp. 1410–1420.
4. Zhao, N.; Chen, J.; Wang, Z.; Peng, X.; Wang, G.; Wu, Y.; Zhou, F.; Feng, Z.; Nie, X.; Zhang, W.; et al. Real-time incident prediction for online service systems. In Proceedings of the 28th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering, Virtual Event, 8–13 November 2020; ACM: New York, NY, USA, 2020; pp. 315–326.
5. Chen, Y.; Yang, X.; Dong, H.; He, X.; Zhang, H.; Lin, Q.; Chen, J.; Zhao, P.; Kang, Y.; Gao, F.; et al. Identifying linked incidents in large-scale online service systems. In Proceedings of the 28th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering, Virtual Event, 8–13 November 2020; ACM: New York, NY, USA, 2020; pp. 304–314.
6. Zhao, N.; Chen, J.; Peng, X.; Wang, H.; Wu, X.; Zhang, Y.; Chen, Z.; Zheng, X.; Nie, X.; Wang, G.; et al. Understanding and handling alert storm for online service systems. In Proceedings of the ACM/IEEE 42nd International Conference on Software Engineering: Software Engineering in Practice, Seoul, Republic of Korea, 27 June–19 July 2020; ACM: New York, NY, USA, 2020; pp. 162–171.
7. Ma, M.; Yin, Z.; Zhang, S.; Wang, S.; Zheng, C.; Jiang, X.; Hu, H.; Luo, C.; Li, Y.; Qiu, N.; et al. Diagnosing root causes of intermittent slow queries in cloud databases. *Proc. VLDB Endow.* **2020**, *13*, 1176–1189. [CrossRef]
8. Zhu, J.; He, S.; He, P.; Liu, J.; Lyu, M.R. Loghub: A large collection of system log datasets for ai-driven log analytics. In Proceedings of the 2023 IEEE 34th International Symposium on Software Reliability Engineering (ISSRE), Florence, Italy, 9–12 October 2023; IEEE: Piscataway, NJ, USA, 2023; pp. 355–366.
9. Zhang, X.; Xu, Y.; Lin, Q.; Qiao, B.; Zhang, H.; Dang, Y.; Xie, C.; Yang, X.; Cheng, Q.; Li, Z.; et al. Robust log-based anomaly detection on unstable log data. In Proceedings of the 2019 27th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering, Tallinn, Estonia, 26–30 August 2019; pp. 807–817.
10. Bodik, P.; Goldszmidt, M.; Fox, A.; Woodard, D.B.; Andersen, H. Fingerprinting the datacenter: Automated classification of performance crises. In Proceedings of the 5th European Conference on Computer Systems, Paris, France, 13–16 April 2010; pp. 111–124.
11. Chen, M.; Zheng, A.X.; Lloyd, J.; Jordan, M.I.; Brewer, E. Failure diagnosis using decision trees. In Proceedings of the International Conference on Autonomic Computing, 2004. Proceedings, New York, NY, USA, 17–18 May 2004; IEEE: Piscataway, NJ, USA, 2004; pp. 36–43.
12. Babenko, A.; Mariani, L.; Pastore, F. AVA: Automated interpretation of dynamically detected anomalies. In Proceedings of the Eighteenth International Symposium on Software Testing and Analysis, Chicago, IL, USA, 19–23 July 2009; pp. 237–248.
13. Jia, T.; Li, Y.; Yang, Y.; Huang, G.; Wu, Z.Augmenting Log-based Anomaly Detection Models to Reduce False Anomalies with Human Feedback. In Proceedings of the 28th ACM SIGKDD Conference on Knowledge Discovery and Data Mining, Washington, DC, USA, 14–18 August 2022; pp. 3081–3089.

14. Fronza, I.; Sillitti, A.; Succi, G.; Terho, M.; Vlasenko, J. Failure prediction based on log files using random indexing and support vector machines. *J. Syst. Softw.* **2013**, *86*, 2–11. [CrossRef]
15. Liang, Y.; Zhang, Y.; Xiong, H.; Sahoo, R. Failure prediction in ibm bluegene/l event logs. In Proceedings of the Seventh IEEE International Conference on Data Mining (ICDM 2007), Omaha, NE, USA, 28–31 October 2007; IEEE: Piscataway, NJ, USA, 2007; pp. 583–588.
16. Lu, S.; Wei, X.; Li, Y.; Wang, L. Detecting anomaly in big data system logs using convolutional neural network. In Proceedings of the 2018 IEEE 16th Intl Conf on Dependable, Autonomic and Secure Computing, 16th Intl Conf on Pervasive Intelligence and Computing, 4th Intl Conf on Big Data Intelligence and Computing and Cyber Science and Technology Congress (DASC/PiCom/DataCom/CyberSciTech), Athens, Greece, 12–15 August 2018; IEEE: Piscataway, NJ, USA, 2018; pp. 151–158.
17. Du, M.; Li, F.; Zheng, G.; Srikumar, V. Deeplog: Anomaly detection and diagnosis from system logs through deep learning. In Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security, Dallas, TX, USA, 30 October–3 November 2017; pp. 1285–1298.
18. Abdi, H.; Williams, L.J. Principal component analysis. *Wiley Interdiscip. Rev. Comput. Stat.* **2010**, *2*, 433–459. [CrossRef]
19. Guo, H.; Yuan, S.; Wu, X. Logbert: Log anomaly detection via bert. In Proceedings of the 2021 International Joint Conference on Neural Networks (IJCNN), Shenzhen, China, 18–22 July 2021; IEEE: Piscataway, NJ, USA, 2021; pp. 1–8.
20. Zhang, C.; Jia, T.; Shen, G.; Zhu, P.; Li, Y. MetaLog: Generalizable Cross-System Anomaly Detection from Logs with Meta-Learning. In Proceedings of the IEEE/ACM 46th International Conference on Software Engineering (ICSE), Lisbon, Portugal, 14–20 April 2024; pp. 1–12.
21. Han, X.; Yuan, S. Unsupervised cross-system log anomaly detection via domain adaptation. In Proceedings of the 30th ACM International Conference on Information and Knowledge Management, Virtual Event, 1–5 November 2021; pp. 3068–3072.
22. Farahani, A.; Voghoei, S.; Rasheed, K.; Arabnia, H.R. A brief review of domain adaptation. In *Proceedings of the Advances in Data Science and Information Engineering, Las Vegas, NV, USA, 27–30 July 2020*; Proceedings from ICDATA 2020 and IKE 2020; Springer: Cham, Switzerland, 2021; pp. 877–894.
23. Landauer, M.; Onder, S.; Skopik, F.; Wurzenberger, M. Deep learning for anomaly detection in log data: A survey. *Mach. Learn. Appl.* **2023**, *12*, 100470. [CrossRef]
24. Yin, K.; Yan, M.; Xu, L.; Xu, Z.; Li, Z.; Yang, D.; Zhang, X. Improving log-based anomaly detection with component-aware analysis. In Proceedings of the 2020 IEEE International Conference on Software Maintenance and Evolution (ICSME), Adelaide, SA, Australia, 28 September–2 October 2020; IEEE: Piscataway, NJ, USA, 2020; pp. 667–671.
25. Meng, W.; Liu, Y.; Zhu, Y.; Zhang, S.; Pei, D.; Liu, Y.; Chen, Y.; Zhang, R.; Tao, S.; Sun, P.; et al. Loganomaly: Unsupervised detection of sequential and quantitative anomalies in unstructured logs. In Proceedings of the IJCAI (International Joint Conference on Artificial Intelligence), Macao, China, 10–16 August 2019; Volume 19, pp. 4739–4745.
26. Wittkopp, T.; Acker, A.; Nedelkoski, S.; Bogatinovski, J.; Scheinert, D.; Fan, W.; Kao, O. A2log: Attentive augmented log anomaly detection. *arXiv* **2021**, arXiv:2109.09537.
27. Chen, R.; Zhang, S.; Li, D.; Zhang, Y.; Guo, F.; Meng, W.; Pei, D.; Zhang, Y.; Chen, X.; Liu, Y. Logtransfer: Cross-system log anomaly detection for software systems with transfer learning. In Proceedings of the 2020 IEEE 31st International Symposium on Software Reliability Engineering (ISSRE), Coimbra, Portugal, 12–15 October 2020; IEEE: Piscataway, NJ, USA, 2020; pp. 37–47.
28. Yang, L.; Chen, J.; Wang, Z.; Wang, W.; Jiang, J.; Dong, X.; Zhang, W. Semi-supervised log-based anomaly detection via probabilistic label estimation. In Proceedings of the 2021 IEEE/ACM 43rd International Conference on Software Engineering (ICSE), Madrid, Spain, 22–30 May 2021; IEEE: Piscataway, NJ, USA, 2021; pp. 1448–1460.
29. Du, M.; Li, F. Spell: Streaming parsing of system event logs. In Proceedings of the 2016 IEEE 16th International Conference on Data Mining (ICDM), Barcelona, Spain, 12–15 December 2016; pp. 859–864.
30. He, P.; Zhu, J.; Zheng, Z.; Lyu, M.R. Drain: An online log parsing approach with fixed depth tree. In Proceedings of the 2017 IEEE International Conference on Web Services (ICWS), Honolulu, HI, USA, 25–30 June 2017; IEEE: Piscataway, NJ, USA, 2017; pp. 33–40.
31. Xu, W.; Huang, L.; Fox, A.; Patterson, D.; Jordan, M. Largescale system problem detection by mining console logs. In Proceedings of the SOSP'09: 22nd ACM Symposium on Operating Systems Principles, Big Sky, MT, USA, 11–14 October 2009.
32. Oliner, A.; Stearley, J. What supercomputers say: A study of five system logs. In Proceedings of the 37th Annual IEEE/IFIP International Conference on Dependable Systems and Networks DSN'07, Edinburgh, UK, 25–28 June 2007; pp. 575–584.
33. Litrico, M.; Del Bue, A.; Morerio, P. Guiding pseudo-labels with uncertainty estimation for source-free unsupervised domain adaptation. In Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition, Vancouver, BC, Canada, 17–24 June 2023; pp. 7640–7650.

34. Amersfoort, J.V.; Smith, L.; Teh, Y.W.; Gal, Y. Uncertainty estimation using a single deep deterministic neural network. In Proceedings of the International Conference on Machine Learning, Virtual Event, 3–18 July 2020; JMLR: Norfolk, MA, USA, 2020; pp. 9690–9700.
35. Wang, J.; Zhao, C.; He, S.; Gu, Y.; Alfarraj, O. LogUAD: Log unsupervised anomaly detection based on Word2Vec. *Comput. Syst. Sci. Eng.* **2022**, *41*, 1207. [CrossRef]

Disclaimer/Publisher's Note: The statements, opinions and data contained in all publications are solely those of the individual author(s) and contributor(s) and not of MDPI and/or the editor(s). MDPI and/or the editor(s) disclaim responsibility for any injury to people or property resulting from any ideas, methods, instructions or products referred to in the content.